

Col·leccions

1 Tuples

2. Diccionaris



- Tuplas
- Conjuntos
- Diccionarios

- Pilas
 - Colas
- | simulables con listas

Col·leccions

Una col·lecció permet agrupar diversos objectes sota un mateix nom.

Per exemple, si necessitem emmagatzemar al nostre programa els nombres dels alumnes d'un curs de programació, serà més convenient ubicar-los a tots dins d'una mateixa col·lecció de nom alumnes, al lloc de crear els objectes alumne1, alumne2, etc.

A Python existeixen tres col·leccions bàsiques : les llistes, les tuples i els diccionaris.

Tuples

Una **tupla** és una seqüència ordenada d'objectes de diferents tipus.

Es construeixen posant els elements entre claudàtors () separats per comes.

Es caracteritzen per:

- Tenen ordre.
- Poden contenir elements de diferents tipus.
- Són immutables, és a dir, no es poden alterar durant l'execució d'un programa.

S'utilitzen habitualment per representar col·leccions de dades una determinada estructura semàntica, com ara un vector o una matriu.

`t = (1, 2, "ana")`

```
<class 'tuple'>
```

```
# Tupla amb elements de  
diferents tipus
```

```
(1, "dos", True)
```

```
# Vector
```

```
(1, 2, 3)
```

```
# Matriu
```

```
((1, 2, 3), (4, 5, 6))
```

Creació de tuples mitjançant la funció `tuple()`

Una altra forma de crear tuples és mitjançant la funció `tuple()`.

- `tuple(c)`: Crea una tupla amb els elements de la seqüència o col·lecció `c`.

Podeu indicar els elements separats per comes, mitjançant una cadena, o mitjançant una col·lecció d'elements iterable.

```
>>> tuple()
()
>>> tuple(1, 2, 3)
(1, 2, 3)
>>> tuple("Python")
('P', 'y', 't', 'h', 'o', 'n')
>>> tuple([1, 2, 3])
(1, 2, 3)
```

Operacions amb tuples

L'accés als elements d'una tupla es fa de la mateixa manera que a les llistes. També es poden obtenir subtuples de la mateixa manera que les subllistes.

Les operacions de llistes que no modifiquen la llista també són aplicables a les tuples.

```
>>> a = (1, 2, 3)
>>> a[1]
2
>>> len(a)
3
>>> a.index(3)
2
>>> 0 in a
False
>>> b = ((1, 2, 3), (4, 5, 6))
>>> b[1]
(4, 5, 6)
>>> b[1][2]
6
```

```
In [1]: tupla = (100,"Hola",[1,2,3,4],-50)
```

```
In [2]: tupla
```

```
Out[2]: (100, 'Hola', [1, 2, 3, 4], -50)
```

```
In [3]: tupla[0]
```

```
Out[3]: 100
```

```
In [4]: tupla[-1]
```

```
Out[4]: -50
```

```
In [6]: tupla[2:]
```

```
Out[6]: ([1, 2, 3, 4], -50)
```

Tuplas con índices y slicing

```
In [8]: tupla[2][-1]
```

```
Out[8]: 4
```

```
In [9]: tupla[0] = 50
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-9-b45433b4cee9> in <module>()  
----> 1 tupla[0] = 50  
  
TypeError: 'tuple' object does not support item assignment
```

```
In [10]: len(tupla)
```

```
Out[10]: 4
```

```
In [13]: len(tupla[2])
```

```
Out[13]: 3
```

```
In [15]: # .index para buscar un elemento y saber su posición  
tupla.index(100)
```

```
Out[15]: 0
```

```
In [16]: tupla
```

```
Out[16]: (100, 'Hola', [1, 2, 3], -50)
```

```
In [17]: tupla.index('Hola')
```

```
Out[17]: 1
```

```
In [18]: tupla.index('Otro')
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-18-640d616163a2> in <module>()  
----> 1 tupla.index('Otro')  
  
ValueError: tuple.index(x): x not in tuple
```

len
.index

```
(100, 'Höla', [1, 2, 3], -50)
```

```
In [19]: tupla.count(100)
```

```
Out[19]: 1
```

```
In [20]: tupla.count('Algo')
```

```
Out[20]: 0
```

```
In [21]: tupla = (100, 100, 100, 50, 10)
```

```
In [22]: tupla.count(100)
```

```
Out[22]: 3
```

I

```
In [23]: tupla.append(10)
```

```
-----  
AttributeError                                Traceback (most recent call last)  
<ipython-input-23-758d19Sec9d7> in <module>()  
----> 1 tupla.append(10)  
  
AttributeError: 'tuple' object has no attribute 'append'
```

.count

Se pueden crear tuplas n-dimensionales, pero no se pueden variar.

```
t1 = "08", "Barcelona", "Catalunya"  
t2 = "27", "Girona", "Catalunya"  
todas = t1,t2  
print (todas)
```

```
(('08', 'Barcelona', 'Catalunya'), ('27', 'Girona', 'Catalunya'))
```

Podemos crear dinámicamente una lista

```
t = [i for i in range(5)]  
print (t)
```

Esto funciona con listas

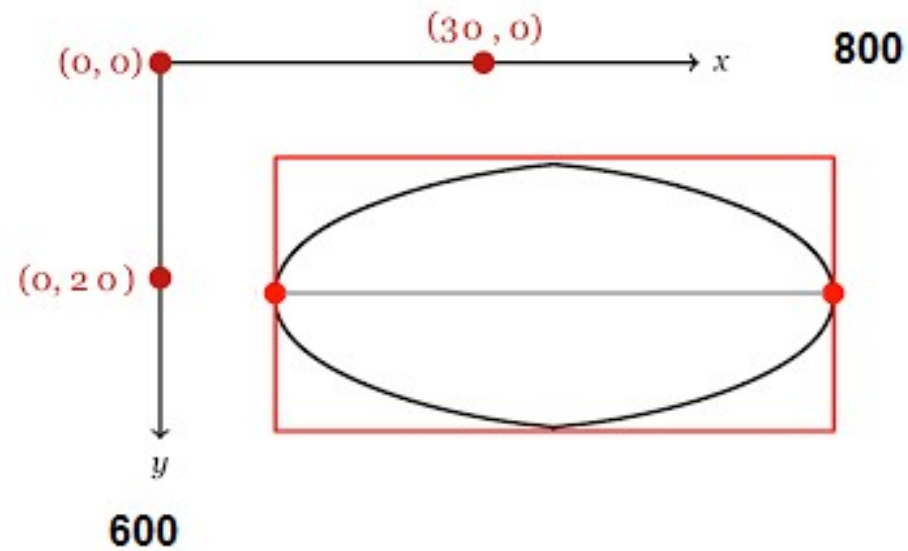
```
[0, 1, 2, 3, 4]
```

Comprueba si funciona con tuplas :

```
t = (i for i in range(5))  
print (t)
```



Prepara un cuaderno jupyter con estas prácticas



Pràctica P01.1: El·lipse

Estudia el següent programa, quantes tuples veus ?

```
from PIL import Image, ImageDraw

img = Image.new('RGB', (600, 400), 'White')
dib = ImageDraw.Draw(img)
#----- dibuja una elipse y la muestra en el lienzo
dib.ellipse((150, 50, 449, 349), 'Crimson') # (x0, y0, x1, y1)
img.show()
#----- salva la imagen y su miniatura
img.save('./img/japon.jpg', "JPEG")
size = (128, 128)
img.thumbnail(size)
img.save('./img/japon.jpg ', "JPEG")
```

Realitza els canvis

1. Mida de la figura original a 500 x 500 color 'darkgoldenrod'
2. Dibuixar una el·lipse a la mateixa posició de l'exercici anterior però color 'Black'
3. Dibuixar una nova el·lipse des de (50,70) a (150, 170) en color 'lavender'
4. Salva els fitxers com [ikusa.jpg](#) i [ikusa-mini.jpg](#)

Práctica P02-Tuplas

1- Tenemos una tupla con datos de una persona :

`t= 89766, "Alicia", "Hacker"`

Muestra los datos y pide año de registro.

Puedes añadir el año a la cola de esta tupla ?

Como puedes conseguir una tupla con los datos de la persona y el año de registro ?

2- Programa la instrucción para mostrar los elementos 2ndo y 3ero

3- **Avanzado** : Crea una tupla tipo `"personas = ((codigo, nombre, apellidos, anyo_reg))"` con los datos de Alicia.

Crea otra tupla de persona con tus datos.

Nota : Usa como "código" -> tu "dni" sin la letra.

Muestra la lista de personas (print)

- 4- Crea un programa temps.py que tenga definida una matriz de tuplas del tipo : "Barcelona", 27, 28, 26, 24, 22
- las columnas son : ubicación y las temperaturas medias de los últimos 5 días.
 - las filas serán 4 ubicaciones geográficas distintas

Pide por pantalla una ubicación y muestra las temperaturas de los 5 días y la media de los 5 valores.

Si la ubicación es = "*" entonces muestra todas las ubicaciones.

- 5- **Avanzado** : perfecciona el programa para que al empezar muestre la lista de ubicaciones disponibles y al comparar las ubicaciones existentes con la introducida sea insensible a mayúsculas y minúsculas. Ver `string.upper()`

Práctica P03.1 – Métodos de las tuplas - Base

Escribir un programa que permita procesar (*) datos de pasajeros de viaje en una lista de tuplas con la siguiente forma:

pasajeros = (nombre, dni, ciudad_destino).

Ejemplo: **pasajeros** =

```
[ ("Manuel Juarez", 19823451, "Liverpool"),  
  ("Silvana Paredes", 22709128, "Buenos Aires"),  
  ("Rosa Ortiz", 15123978, "Glasgow"),  
  ("Luciana Hernandez", 38981374, "Lisboa") ]
```

Además, en otra lista de tuplas se almacenan los datos de cada ciudad y el país al que pertenecen.

Ejemplo: **ciudades** = [("Buenos Aires", "Argentina"),
 ("Glasgow", "Escocia"), ("Lisboa", "Portugal"),
 ("Liverpool", "Inglaterra"), ("Madrid", "España")]

Práctica P03.1 – Métodos de las tuplas - Base

(*) Hacer un menú que permita al usuario realizar las siguientes operaciones:

0-Salir del programa.

1-Agregar pasajero : Agregar pasajeros a la lista de **pasajeros**.

2-Agregar ciudad : Agregar ciudades a la lista de **ciudades**.

sigue

Práctica P03.1 – Métodos de las tuplas - Avanzado

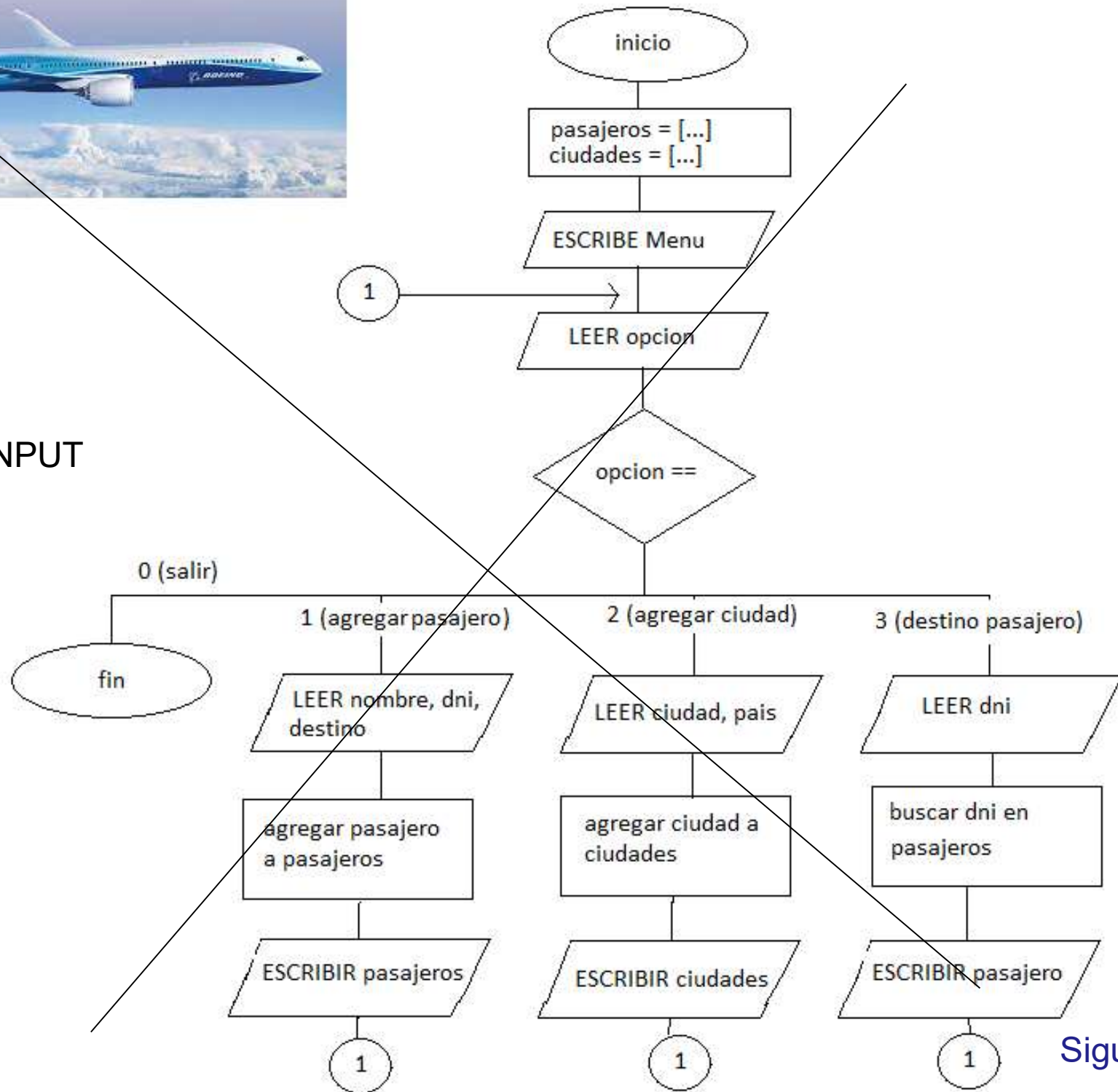
3-Destino - Pasajero: Dado el DNI de un pasajero, ver a qué país y ciudad viaja.

4-Pasajeros x Ciudad : Dada una ciudad, mostrar la cantidad de pasajeros que viajan a esa ciudad.

5-Pasajeros x País : Dado un país, mostrar cuántos pasajeros viajan a ese país.



LEER = INPUT



Sigue...

Práctica P03.2 – Coordenadas

Estudia el siguiente programa, modifícalo para que dibuje un tablero de ajedrez.

```
from PIL import Image, ImageDraw
```

```
w, h = 800, 600
```

```
# -----
```

```
img = Image.new("RGB", (w, h))
```

```
dib = ImageDraw.Draw(img)
```

```
# -----
```

```
for i in range (8):
```

```
    # calcula coordenadas, cuadrados de 100 x 100 pixels
```

```
    col = i*100
```

```
    shape = [(col, 10), (col + 100, 100)]
```

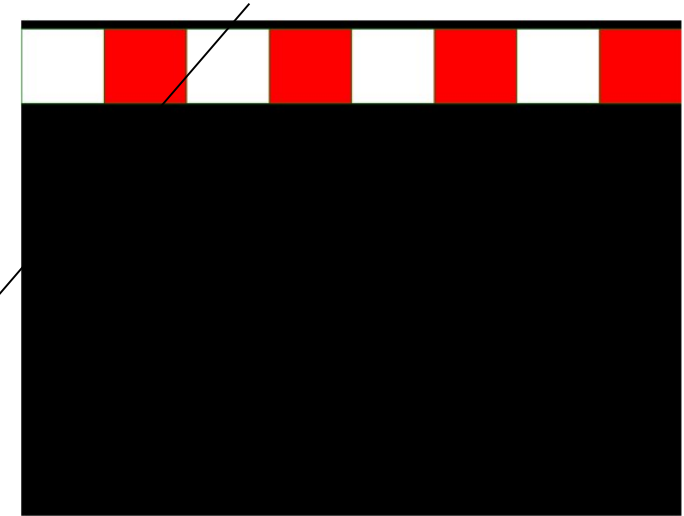
```
    print (shape)
```

```
    if i%2 > 0 : color = 'red'
```

```
    else : color = 'white'
```

```
    dib.rectangle(shape, fill=color , outline ="green")
```

```
img.show()
```



Soluciones

#P01 – Elipse-----

```
from PIL import Image, ImageDraw

img = Image.new('RGB', (500, 500), 'darkgoldenrod')
dib = ImageDraw.Draw(img)

#----- dibuja una elipse y la muestra en el lienzo
dib.ellipse((150, 50, 449, 349), 'Black')      # (x0,y0, x1, y1)
dib.ellipse((70, 50, 170, 150), 'lavender')     # (x0,y0, x1, y1)
img.show()

#----- salva la imagen y su miniatura
img.save('img\ikusa.jpg', "JPEG")
size = (128, 128)
img.thumbnail(size)
img.save('img\ikusa-mini.jpg', "JPEG")
```

Visita estos enlaces,

<https://pillow.readthedocs.io/en/stable/handbook/tutorial.html#create-jpeg-thumbnails>

Como crear todas las miniaturas de una lista de imágenes pasada como argumento.

<https://www.codegrepper.com/code-examples/python/plotly+colours>

Soluciones

#P02 - Tuplas-----

#1.-

```
t= (89766, "Alicia", "Hacker")
any_reg = int(input("Año registro : "))
ta = t, any_reg
print (ta)
```

#P02-----

#2.-

```
print(ta[0][1], ta[0][2])
```

#P02 - Tuplas-----

#3.-

```
p1 = (89766, "Alicia", "Hacker", 2010 )
p2 = (46227578, "Marc", " Martín Rus ", 2008)
```

```
persones = p1, p2
for p in persones :
    print (p)
```

Soluciones

#P02 Exercici 5 -----

```
u1 = "Barcelona", 27, 28, 26, 24, 22
u2 = "Tarragona", 29, 30, 29, 30, 28
u3 = "Lleida", 22, 22, 21, 20, 21
u4 = "Girona", 26, 25, 24, 22, 20
```

```
ubicacions = (u1,u2,u3,u4)
```

```
print("Ubicacions disponibles")
```

```
print("-----")
```

```
for t in ubicacions :
    print (t[0])
```

```
ubicacio = input("Ubicació a consultar : ")
```

```
ubicacio = ubicacio .upper()
```

```
for t in ubicacions :
    if t[0].upper() == ubicacio or ubicacio == "*" :
        print (t)
```

#P02 Exercici 4 -----

```
u1 = "Barcelona", 27, 28, 26, 24, 22
u2 = "Tarragona", 29, 30, 29, 30, 28
u3 = "Lleida", 22, 22, 21, 20, 21
u4 = "Girona", 26, 25, 24, 22, 20
```

```
ubicacions = (u1,u2,u3,u4)
```

```
ubicacio = input("Ubicació : ")
```

```
for t in ubicacions :
    if t.count(ubicacio ) > 0
        or ubicacio == "*" :
        print (t)
```

#solución P03-2 Coordenadas

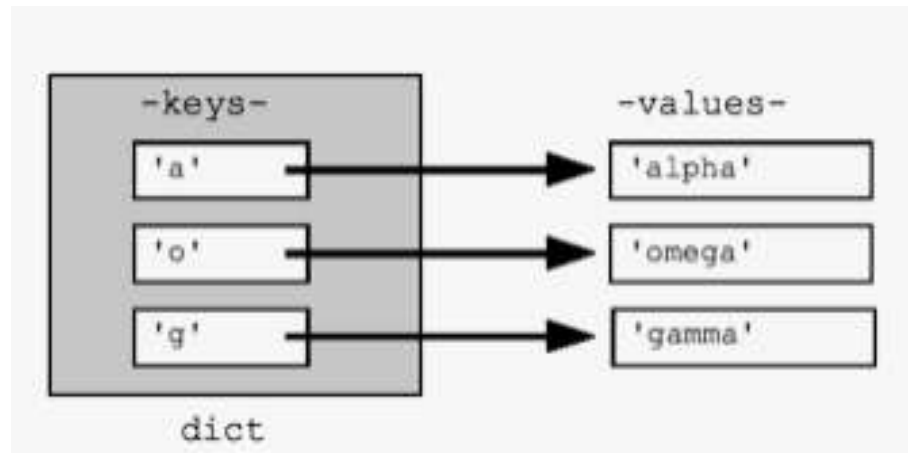
```
from PIL import Image, ImageDraw

w, h = 802, 802
#-----
img = Image.new("RGB", (w, h))
dib = ImageDraw.Draw(img)
#-----

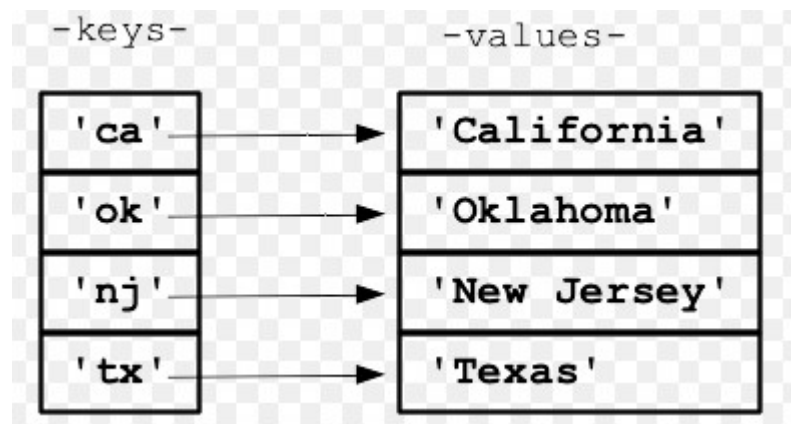
for linea in range (8):
    for i in range (8):
        # calcula coordenadas, cuadrados de 100 x 100 pixels
        col = i*100
        lin = linea * 100
        shape = [(col, lin), (col + 100, lin + 100)]
        print (shape)
        if linea % 2 > 0 :
            if i%2 > 0 : color = 'black'
            else : color = 'white'
        else:
            if i%2 > 0 : color = 'white'
            else : color = 'black'
        dib.rectangle(shape, fill=color , outline ="green")

img.show()
```

Diccionaris



```
{"Name": "John",  
 "Age": 21,  
 "Id": 28}
```



```
{  
  'name' : ['Joe', 'Nat', 'Harry'],  
  'age'   : [20, 21, 19],  
  'marks' : [85.1, 77.8, 91.54]  
}
```

	name	age	marks
0	Joe	20	85.1
1	Nat	21	77.8
2	Harry	19	91.54

Diccionaris

Un diccionari és una col·lecció de parells formats per una *clau* i un *valor* associat a la clau.

Es construeixen posant els parells entre claus `{ }` separats per comes, i separant la clau del valor amb dos punts `:`.

`d = { 'c': 'capità', 'm': 'mariner', }`

Es caracteritzen per:

- No tenen cap ordre.
- Poden contenir elements de diferents tipus.
- Són mutables, és a dir, es poden alterar durant l'execució d'un programa.
- Les claus són úniques, és a dir, no es poden repetir en un mateix diccionari, i poden ser de qualsevol tipus de dades immutable.

```
In [1]: vacio = {}
```

```
numeros = {10:'diez',20:'veinte'}
```

```
In [2]: vacio
```

```
numeros[10]
```

```
Out[2]: {}
```

```
'diez'
```

```
In [3]: type(vacio)
```

```
Out[3]: dict
```

```
In [4]: colores = {'amarillo':'yellow','azul':'blue', 'verde':'green'}
```

```
In [5]: colores
```

```
Out[5]: {'amarillo': 'yellow', 'azul': 'blue', 'verde': 'green'}
```

```
In [6]: colores['azul']
```

```
Out[6]: 'blue'
```

```
In [7]: colores['amarillo']
```

```
Out[7]: 'yellow'
```

```
colores['amarillo'] = 'white'
```

```
colores
```

```
{'amarillo': 'white', 'azul': 'blue', 'verde': 'green'}
```

```
edades = {'Hector':27, 'Juan':45, 'Maria':34}
print (edades ['Hector'])
```

```
edades_pers
```

```
{'Hector': 27, 'Juan': 45, 'Maria': 34}
```

```
edades_pers ['Hector'] +=1
edades_pers
```

```
{'Hector': 28, 'Juan': 45, 'Maria': 34}
```

```
edades_pers ['Juan'] + edades_pers ['Maria']
```

```
79
```

```
for p in edades_pers :
    print (p)
```

```
Hector
Juan
Maria
```

```
for clave in edades_pers :
    print (edades_pers[clave])
```

```
28
45
34
```

```
for clave in edades_pers :  
    print (clave, edades_pers[clave])
```

Hector 28
Juan 45
Maria 34

```
for clave, valor in edades_pers.items() :  
    print (clave, valor)
```

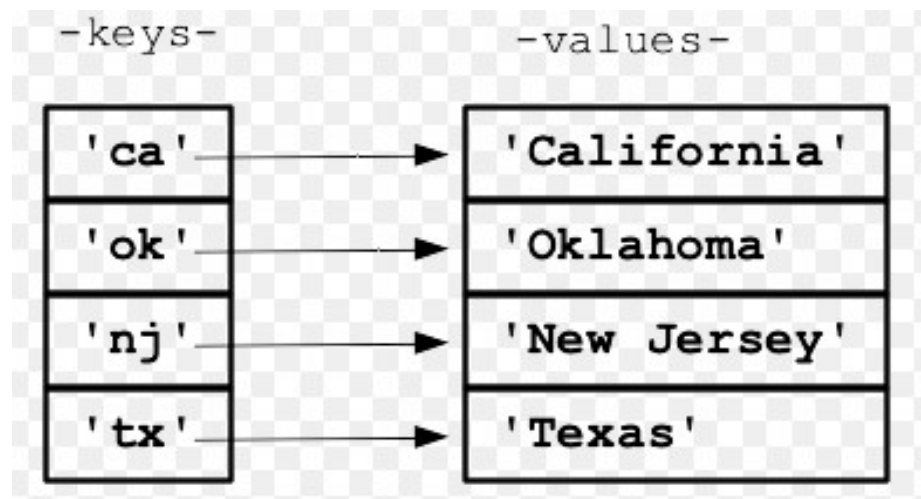
Hector 28
Juan 45
Maria 34

```
for c,v in edades_pers.items() :  
    print (c,v)
```

Hector 28
Juan 45
Maria 34

Exercici 1 : Crea un diccionari per contenir els valors d'aquestes taules. I Completa aquests programes:

estats



```
estat = input ("codi estat")  
print ("El nom del estat és : ")
```

frutas o fruites

Fruta	Precio
Plátano	1.35
Manzana	0.80
Pera	0.85
Naranja	0.70

```
fruta = input ("Introdueixi una fruta : ")  
print ("El seu preu és : ")
```

Exemples d'ús

Ús del diccionari per desar registres amb diversos camps :

```
{'host': '197.109.77.178 ', 'user_name': 'kertzmann3129', 'time': ''}  
{'host': '156.127.178.177 ', 'user_name': 'okuneva5222', 'time': ''}  
{'host': '100.32.205.59 ', 'user_name': 'ortiz8891', 'time': ''}  
{'host': '168.95.156.240 ', 'user_name': 'stark2413', 'time': ''}  
{'host': '71.172.239.195 ', 'user_name': 'dooley1853', 'time': ''}  
{'host': '180.95.121.94 ', 'user_name': 'mohr6893', 'time': ''}  
{'host': '144.23.247.108 ', 'user_name': 'auer7552', 'time': ''}
```

Llista de diccionaris:

```
persones = [{ 'nom': 'Hector', 'edat': 27 }, \  
             { 'nom': 'Juan', 'edat': 45 }, \  
             { 'nom': 'Maria', 'edat': 34 }]
```

```
for p in persones :  
    print (p['nom'], p['edat'])
```

Exercici 2 : Personatges Jocs



```
personajes = []

gandalf = {'Nombre': 'Gandalf', 'Clase': 'Mago', 'Raza': 'Humano'}
legolas = {'Nombre': 'Legolas', 'Clase': 'Arquero', 'Raza': 'Elfo'}
gimli = {'Nombre': 'Gimli', 'Clase': 'Guerrero', 'Raza': 'Enano'}

personajes.append(gandalf)
personajes.append(legolas)
personajes.append(gimli)

print(personajes)

[{'Clase': 'Mago', 'Nombre': 'Gandalf', 'Raza': 'Humano'},
 {'Clase': 'Arquero', 'Nombre': 'Legolas', 'Raza': 'Elfo'},
 {'Clase': 'Guerrero', 'Nombre': 'Gimli', 'Raza': 'Enano'}]
```

```
In [24]: personajes = []
```

```
In [25]: p = {'Nombre': 'Gandalf', 'Clase': 'Mago', 'Raza': 'Humano'}
```

```
In [26]: personajes.append(p)
```

```
In [27]: personajes
```

```
Out[27]: [{'Clase': 'Mago', 'Nombre': 'Gandalf', 'Raza': 'Humano'}]
```

```
In [28]: p = {'Nombre': 'Legolas', 'Clase': 'Arquero', 'Raza': 'Elfo'}
```

```
In [29]: personajes.append(p)
```

```
In [30]: p = {'Nombre': 'Gimli', 'Clase': 'Guerrero', 'Raza': 'Enano'}
```

```
In [31]: personajes.append(p)
```

```
In [32]: personajes
```

```
Out[32]: [{'Clase': 'Mago', 'Nombre': 'Gandalf', 'Raza': 'Humano'},  
          {'Clase': 'Arquero', 'Nombre': 'Legolas', 'Raza': 'Elfo'},  
          {'Clase': 'Guerrero', 'Nombre': 'Gimli', 'Raza': 'Enano'}]
```

```
In [33]: for p in personajes:  
         print(p['Nombre'],p['Clase'],p['Raza'])
```

```
Gandalf Mago Humano  
Legolas Arquero Elfo  
Gimli Guerrero Enano
```

Exercici : Afegeix un altre personatge.

Práctica P04 – Personajes

Durante el desarrollo de un pequeño videojuego se te encarga configurar y balancear cada clase de personaje jugable. Partiendo que la estadística base es 2, debes cumplir las siguientes condiciones:

- # Caballero : doble vida y defensa que el guerrero
- # Guerrero : doble ataque y alcance que un caballero
- # Arquero = Misma vida y ataque que un guerrero,
- # pero la mitad de su defensa y el doble de su alcance

Muestra las propiedades de los 3 personajes.

```
caballero = {'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}  
guerrero = {'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}  
arquero = {'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}
```

```
caballero['vida'] = guerrero ['vida'] * 2  
caballero['defensa'] = guerrero ['defensa'] * 2  
print (caballero)
```

```
{'vida': 4, 'ataque': 2, 'defensa': 4, 'alcance': 2}
```

Avanzado:

- Modifica el diccionario añadiendo el nombre del personaje

`{'nombre':'caballero', 'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2} ... etc`

- Crea un programa que permita escoger un personaje (c=caballero, g=guerrero)
- Si han escogido caballero , pide el valor numérico de ataque y alcance.
- Si han escogido guerrero, pide el valor numérico de vida y defensa.
- Calcula el resto de los atributos de todos los personajes.
- Muestra los atributos de todos los personajes

EJEMPLO

Escoja personaje c=caballero, g=guerrero: g

Entre la vida: 8

Entre la defensa : 9

`{'nombre': 'caballero', 'vida': 16.0, 'ataque': 2, 'defensa': 18.0, 'alcance': 2}`

`{'nombre': 'guerrero', 'vida': 8.0, 'ataque': 4, 'defensa': 9.0, 'alcance': 4}`

`{'nombre': 'arquero', 'vida': 8.0, 'ataque': 4, 'defensa': 4.5, 'alcance': 8}`

Estudiem aquest exemple :

Práctica P05 – Diccionaris

```
import random
aleatoris = ["dilluns", "dimarts", "dimecres",
             "dijous", "divendres",
             "dissabte", "diumenge"]
random.shuffle(aleatoris)

print (aleatoris)

dic_fr = {"dilluns": "lundi",
          "dimarts": "mardi", "dimecres": "mercredi",
          "dijous": "jeudi", "divendres": "vendredi",
          "dissabte": "samedi", "diumenge": "dimanche"}

for d in aleatoris :
    print (f" el dia {d:10} traduït és {dic_fr[d]} ")
```

Prova si funciona bé.

I ara continua amb la pràctica següent.

Práctica P05.1 – Diccionarios Idiomas

Crea 3 diccionarios con los días de la semana en catalán/castellano, inglés y francés. Decide cual es el idioma clave y usa el mismo en todos los diccionarios.

- Pide por pantalla el idioma (0= salir).
- Muestra 3 días de la semana escogidos al azar en el idioma solicitado

Métodos de los diccionarios

Tabla de contenidos

get()

keys()

values()

items()

pop()

clear()

```
colores = { "amarillo":"yellow", "azul":"blue", "verde":"green" }
```

get()

Busca un elemento a partir de su clave y si no lo encuentra devuelve un valor por defecto:

Código	Resultado
--------	-----------

```
colores.get('negro','no se encuentra')
```

Method	Description
<u>clear()</u>	Elimina todos los elementos del diccionario
<u>copy()</u>	Devuelve una copia del diccionario
<u>fromkeys(keys,value)</u>	Devuelve un diccionario creado a partir de una lista de claves y con un valor por defecto.
<u>get(key)</u>	Devuelve el valor de una clave, equivale a dict['key']
<u>items()</u>	Devuelve una lista de elementos en forma de tupla (clave, valor)
<u>keys()</u>	Devuelve una lista con las claves del diccionario
<u>pop(key)</u>	Quita el elemento con la clave especificada
<u>popitem()</u>	Devuelve y elimina la clave y valor insertado en ultimo lugar
<u>setdefault(keyname,value)</u>	Devuelve el valor de una clave especificada. Si no existe devuelve un valor por defecto.
<u>update(iterable)</u>	Actualiza el diccionario con los pares clave-valor especificados en un iterable.
<u>values()</u>	Devuelve una lista con los valores del diccionario

Practica P06 – Métodos de los diccionarios

Ejercicio 3

Escribir un programa que :

- tenga en un diccionario los precios de las frutas de la tabla,
- pida al usuario una fruta y un número de kilos
- y muestre por pantalla el importe = precio * número de kilos de fruta.
- Si la fruta no está en el diccionario debe mostrar el mensaje “Producto inexistente”

Fruta	Precio
Plátano	1.35
Manzana	0.80
Pera	0.85
Naranja	0.70

```
{'plátano': 1.35, 'Manzana': 0.8}
seleccione fruta : plátano
cuantos kilos quiere ? : 3.5
A pagar : 4.73
```

Práctica P07 – Diccionarios volando

Adapta el ejercicio P03.1 a diccionarios.

```
pasajeros = [{'nombre':"Manuel Juarez", 'dni':"3333-J", 'ciudad':"Liverpool"},  
             {'nombre':"Silvana Paredes", 'dni':"4444-K", 'ciudad':"Buenos Aires"},  
             {'nombre':"Rosa Ortiz", 'dni':"2222-B", 'ciudad':"Glasgow"},  
             {'nombre':"Luciana Hernandez", 'dni':"1111-A", 'ciudad':"Lisboa"}]
```

```
ciudades = [ {'ciudad':"Buenos Aires", 'pais':"Argentina"},  
             {'ciudad':"Glasgow", 'pais':"Escocia"},  
             {'ciudad':"Lisboa", 'pais':"Portugal"},  
             {'ciudad':"Liverpool", 'pais':"UK"},  
             {'ciudad':"Madrid", 'pais':"España"}]
```

Y adapta el menú que permita al usuario realizar las siguientes operaciones:

0-Salir del programa.

1-Agregar pasajero : Agregar pasajeros a la lista de pasajeros.

2-Agregar Ciudad : Agregar ciudades a la lista de ciudades

3-Destino - Pasajero Dado el DNI de un pasajero, ver a qué país y ciudad viaja.

4-Pasajeros x Ciudad : Dada una ciudad, mostrar la cantidad de pasajeros que viajan a esa ciudad.

5-Pasajeros x País : Dado un país, mostrar cuántos pasajeros viajan a ese país..

Práctica R03 – Métodos de los diccionarios

<https://aprendeconalf.es/docencia/python/manual/diccionarios/>

Práctica R04 – Practica con bucles while

Solución Práctica P04 – Personajes

```
# Práctica P04 – Personajes
# Caballero : doble vida y defensa que el guerrero
# Guerrero : doble ataque y alcance que un caballero
# Arquero = Misma vida y ataque que un guerrero,
#           pero la mitad de su defensa y el doble de su alcance
# Muestra las propiedades de los 3 personajes.
```

```
caballero = {'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}
guerrero = {'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}
arquero = {'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}
```

```
caballero ['vida'] = guerrero ['vida'] * 2
caballero ['defensa'] = guerrero ['defensa'] * 2
```

```
guerrero ['ataque'] = caballero ['ataque'] * 2
guerrero ['alcance'] = caballero ['alcance'] * 2
```

```
arquero ['vida'] = guerrero ['vida']
arquero ['ataque'] = guerrero ['ataque']
arquero ['defensa'] = guerrero ['defensa'] / 2
arquero ['alcance'] = guerrero ['alcance'] * 2
```

```
print (caballero)
print (guerrero)
print (arquero)
```

Solución Práctica P04 – Personajes (Avanzado)

```
# - Modifica el diccionario añadiendo el nombre del personaje
#----- inicio
caballero = {'nombre': 'caballero', 'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}
guerrero = {'nombre': 'guerrero', 'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}
arquero = {'nombre': 'arquero', 'vida':2 , 'ataque':2, 'defensa':2, 'alcance':2}

# Pide un personaje (c=caballero, g=guerrero)
personaje = input ("Escoja personaje c=caballero, g=guerrero: ")

if personaje == "c" :
    # Si han escogido caballero , pide el valor de ataque y alcance.

    caballero['alcance'] = float (input("Entre el alcance: "))
    caballero['ataque'] = float (input("Entre el ataque : "))

elif personaje == "g" :
    # Si han escogido guerrero, pide el valor de vida y defensa.

    guerrero['vida'] = float (input("Entre la vida: "))
    guerrero['defensa'] = float (input("Entre la defensa : "))
```

```
ps = [caballero, guerrero, arquero]    # lista de personajes
```

```
# Calcula el resto de los atributos de todos los personajes.
```

```
# --> aqui llamar a la función de recalcular
```

```
recalcular (caballero, guerrero, arquero)
```

```
# Muestra los atributos de todos los personajes
```

```
for p in ps :
```

```
    print (p)
```

Esta función se coloca al principio
de todo

```
def recalcular (caballero, guerrero, arquero) :
```

```
    caballero ['vida'] = guerrero ['vida'] * 2
```

```
    caballero ['defensa'] = guerrero ['defensa'] * 2
```

```
    guerrero ['ataque'] = caballero ['ataque'] * 2
```

```
    guerrero ['alcance'] = caballero ['alcance'] * 2
```

```
    arquero ['vida'] = guerrero ['vida']
```

```
    arquero ['ataque'] = guerrero ['ataque']
```

```
    arquero ['defensa'] = guerrero ['defensa'] / 2
```

```
    arquero ['alcance'] = guerrero ['alcance'] * 2
```

Solución Práctica P07 –

```
#----- Inicio del programa
pasajeros = [{'nombre':"Manuel Juarez",      'dni':"3333-J", 'ciudad':"Liverpool"},
              {'nombre':"Silvana Paredes",   'dni':"4444-K", 'ciudad':"Buenos Aires"},
              {'nombre':"Rosa Ortiz",        'dni':"2222-B", 'ciudad':"Glasgow"},
              {'nombre':"Luciana Hernandez", 'dni':"1111-A", 'ciudad':"Lisboa"}]

ciudades = [ {'ciudad':"Buenos Aires", 'pais':"Argentina"},
              {'ciudad':"Glasgow",      'pais':"Escocia"},
              {'ciudad':"Lisboa",       'pais':"Portugal"},
              {'ciudad':"Liverpool",   'pais':"UK"},
              {'ciudad':"Madrid",       'pais':"España"}]

menu = "1.Agregar pasajero, 2.Agregar Ciudad, 0=Salir"
opcion = ""

while opcion != "0" :
    print (menu)
    opcion = input("opcion: ")
    if opcion == "1" :
        nombre = input("nombre: ")
        dni     = input("dni: ")
        ciudad  = input("ciudad: ")

        pasajeros.append({'nombre':nombre, 'dni':dni, 'ciudad':ciudad})
        print(pasajeros)
    elif opcion == "2" :
        ciudad = input("ciudad: ")
        pais   = input("pais: ")

        ciudades.append({'ciudad':ciudad, 'pais':pais})
        print( ciudades)
```